MEMORY MANAGEMENT

A Complete Seminar on How Computers Handle Memory

Computer Science | Operating Systems

01 | What is Memory?

Definition

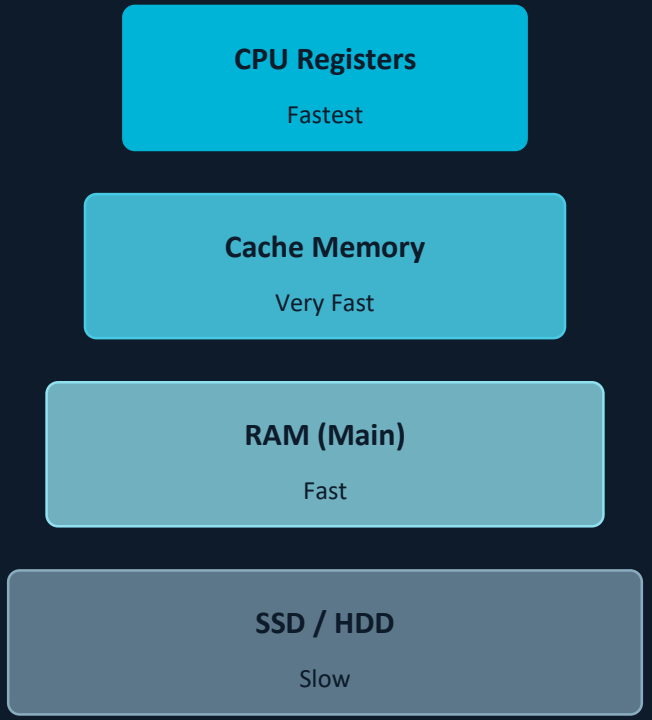
Memory is the part of a computer that stores data and instructions so the CPU can use them.

Real-World Analogy

Think of memory like a desk — RAM is where you keep what you're working on now, and the hard disk is your filing cabinet for finished work.

- Stores currently executing instructions
- Provides fast data access for the CPU
- Holds OS and user programs simultaneously

Memory Hierarchy



02 | What is Memory Management?

Memory management is the process of allocating, tracking, and freeing memory for programs in a computer system.

Allocation

Gives memory to processes.
Decides how much and where in RAM each process is placed.

Deallocation

Frees memory after use.
Returns the block to the free pool for future processes.

Tracking

Keeps record of used & free memory.
Uses tables or bitmaps to track every memory block.

Protection

Prevents one process from accessing another's memory.
Ensures isolation and security.

03 | Why is Memory Management Needed?

✕ WITHOUT Memory Management

Programs may overwrite each other

Process A can corrupt Process B's data.

Memory gets wasted

Unused blocks never freed — memory leaks.

System may crash

Corruption leads to instability and crashes.

✓ WITH Memory Management

Efficient use of memory

Each process gets exactly the memory it needs.

Safe execution of programs

Processes are isolated — crashes don't spread.

Better performance

Faster allocation and smart memory reuse.

04 | Types of Memory

PRIMARY MEMORY

Fast, directly accessible by CPU

RAM — *Random Access Memory*

Stores currently running programs & data
Volatile — data lost on power off

ROM — *Read-Only Memory*

Stores permanent boot instructions (BIOS)
Non-volatile — retained without power

Cache — *Ultra-Fast CPU Buffer*

Extremely fast, located near CPU
Stores frequently accessed data (L1/L2/L3)

SECONDARY MEMORY

Permanent, slower, larger capacity

HDD — *Hard Disk Drive*

Magnetic spinning platters, large capacity (TBs), affordable

SSD — *Solid State Drive*

Flash memory — no moving parts
Much faster than HDD

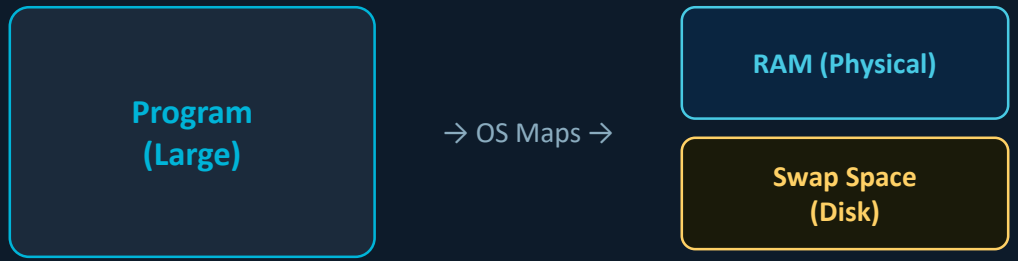
USB — *Pen Drive / USB Flash*

Portable flash storage
Plug-and-play, used for data transfer

05 | Virtual Memory

Virtual memory is a technique that allows execution of large programs by using secondary storage as an extension of main memory. The OS maps virtual addresses to actual RAM.

How Virtual Memory Works



Paging is the technique that makes virtual memory possible.

Each process gets a virtual address space (e.g., 4GB on 32-bit). The OS uses a Page Table to translate virtual → physical RAM addresses. Programs think they have dedicated memory.

▶ Run Large Programs

Execute programs bigger than physical RAM using disk.

▶ Process Isolation

Each process has its own private virtual address space.

▶ Efficient Multitasking

Multiple processes share RAM without interfering.

06 | Paging Technique

Paging divides logical memory into fixed-size pages and physical memory into equal-size frames. The OS maps pages → frames via a Page Table.

✓ Key Benefit: Eliminates external fragmentation — pages don't need to be contiguous in physical memory.

07 | Segmentation

Segmentation is similar to paging, except that the length of segments is variable and pages have a fixed size.

Variable-Length Segments

Each segment corresponds to a logical unit: main function, data structures, utility functions, stack.

Segment Map Table

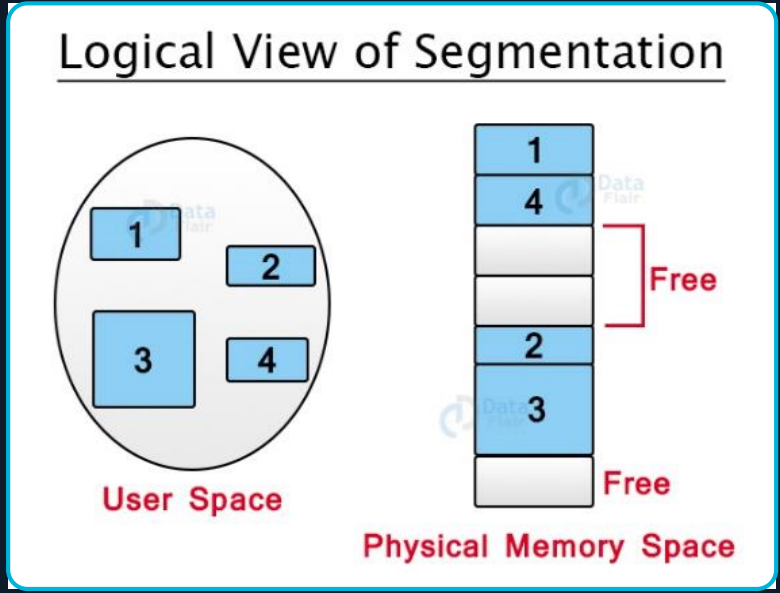
The OS maintains a table with segment number, size, and base address in physical/virtual memory.

User-Visible Division

Unlike paging, the user defines segments — code, data, stack are each a separate segment.

External Fragmentation

Since segments vary in size, free memory may become fragmented into non-contiguous blocks.



Logical View of Segmentation

Paging vs Segmentation

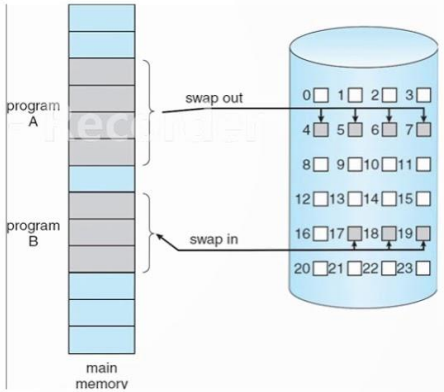
Division: Paging=Fixed-size pages | Segmentation=Variable-size segments

Fragmentation: Paging=Internal | Segmentation=External

09 | Demand Paging

DEMAND PAGING

- **Demand paging** system loads pages only on demand, not in advance
- Instead of swapping the entire process into memory, the pages or the **lazy swapper** is used
- A lazy swapper brings only the necessary pages into memory



Page Table Structure

Valid Bit: 1 = page is in RAM; 0 = page is on disk (causes Page Fault)
Dirty Bit: 1 = page was modified; must be written back to disk before eviction

Key Concepts

Demand Paging

Pages loaded into RAM only when needed (on demand), not in advance. Reduces initial load time.

Lazy Swapper

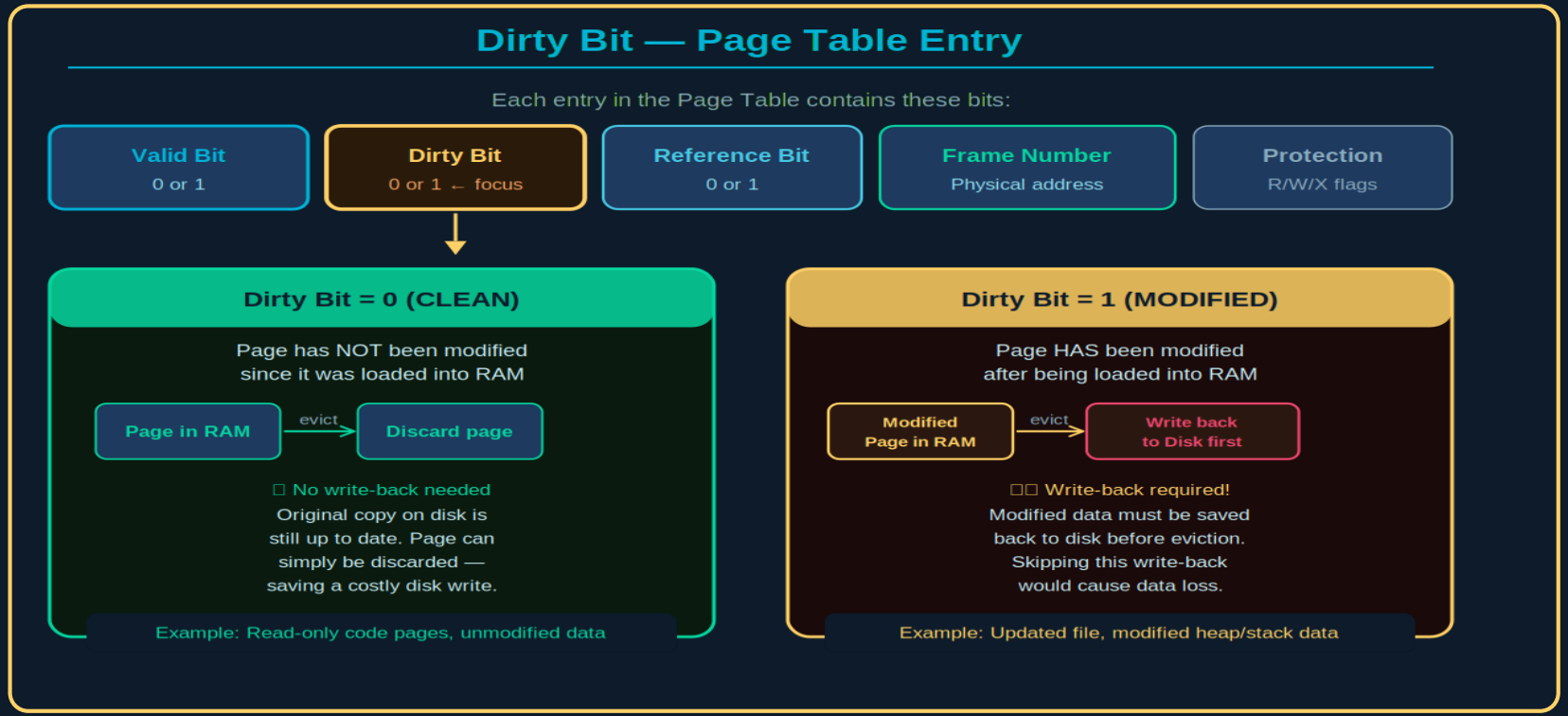
Only the necessary pages are brought into memory — not the entire process at once.

Page Fault

If a required page is not in RAM, a page fault triggers the OS to load it from disk.

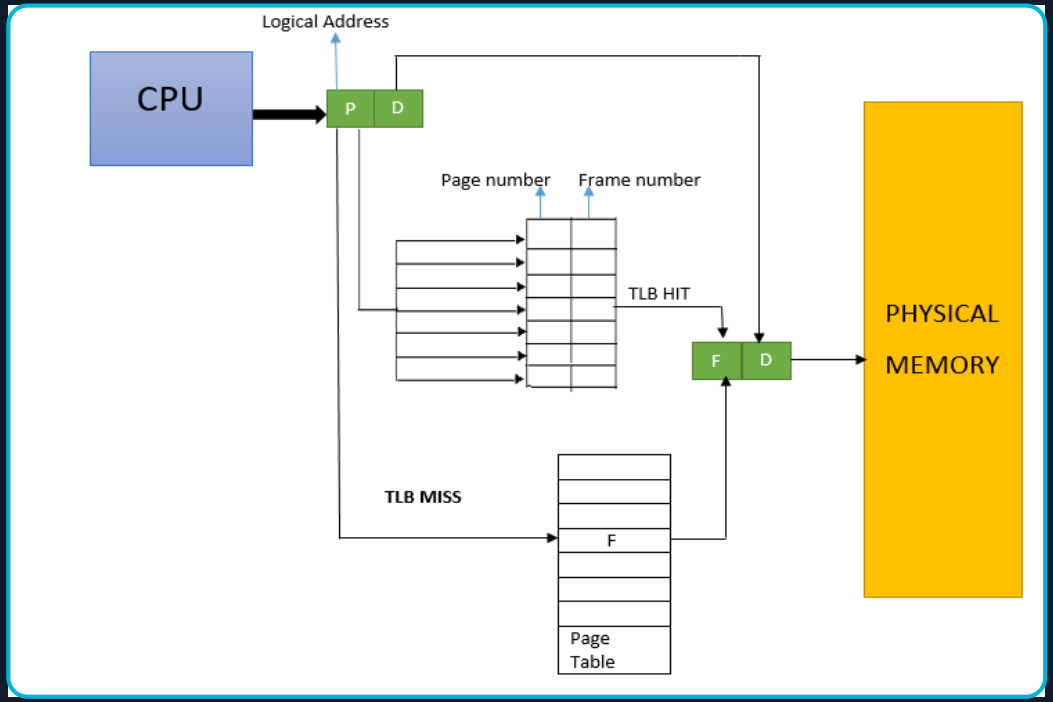
10 | Dirty Bit (Modified Bit)

The Dirty Bit is a flag in the Page Table Entry that tracks whether a page has been modified (written to) since it was loaded into RAM.



11 | TLB — Translation Lookaside Buffer

A TLB is a fast hardware cache inside the CPU that stores recent Page Table entries to speed up virtual-to-physical address translation.



CPU → TLB → Physical Memory Translation

✓ TLB HIT

Virtual address found in TLB → frame number retrieved instantly → access Physical Memory directly.

✗ TLB MISS

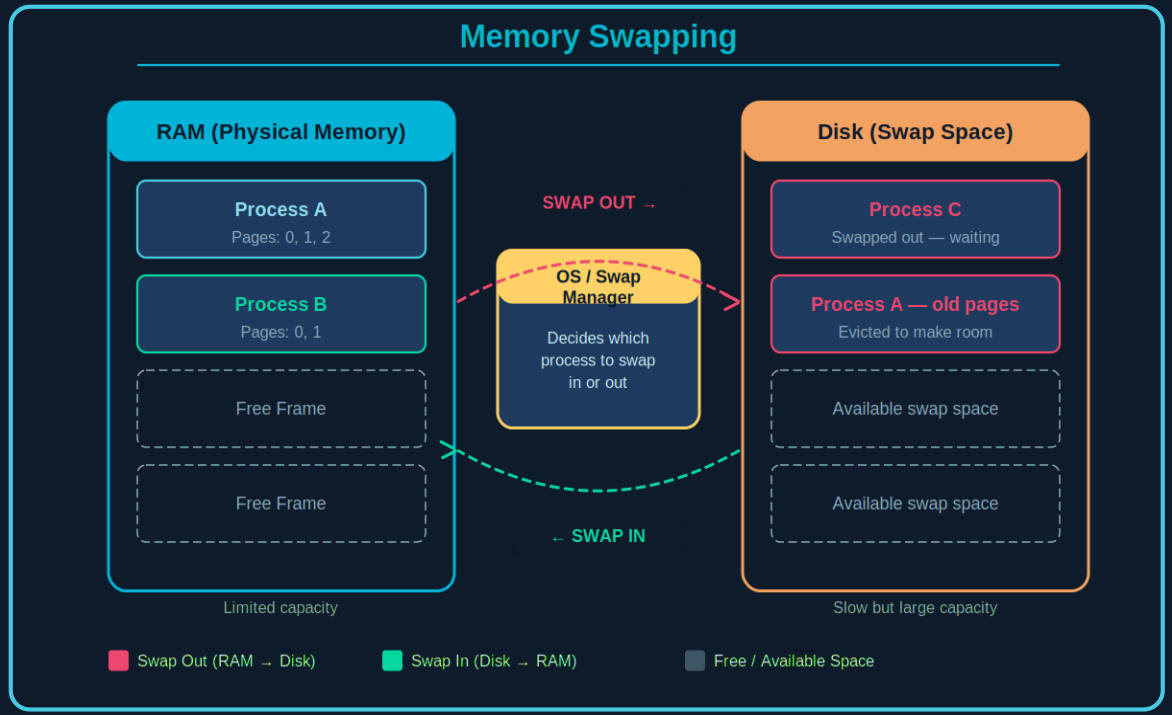
Not found in TLB → OS walks full Page Table → result cached in TLB for future access.

⚡ TLB Performance

Typical size: 64–1024 entries. Hit rates above 98% common. Dramatically reduces address translation overhead.

12 | Swapping

Swapping is when the OS moves an entire process or its pages between RAM and disk to free up memory for other processes.



Key Points

Swap Out

When RAM is full, OS moves a process's pages from RAM → Disk to free space.

Swap In

When process resumes, its pages load back from Disk → RAM for execution.

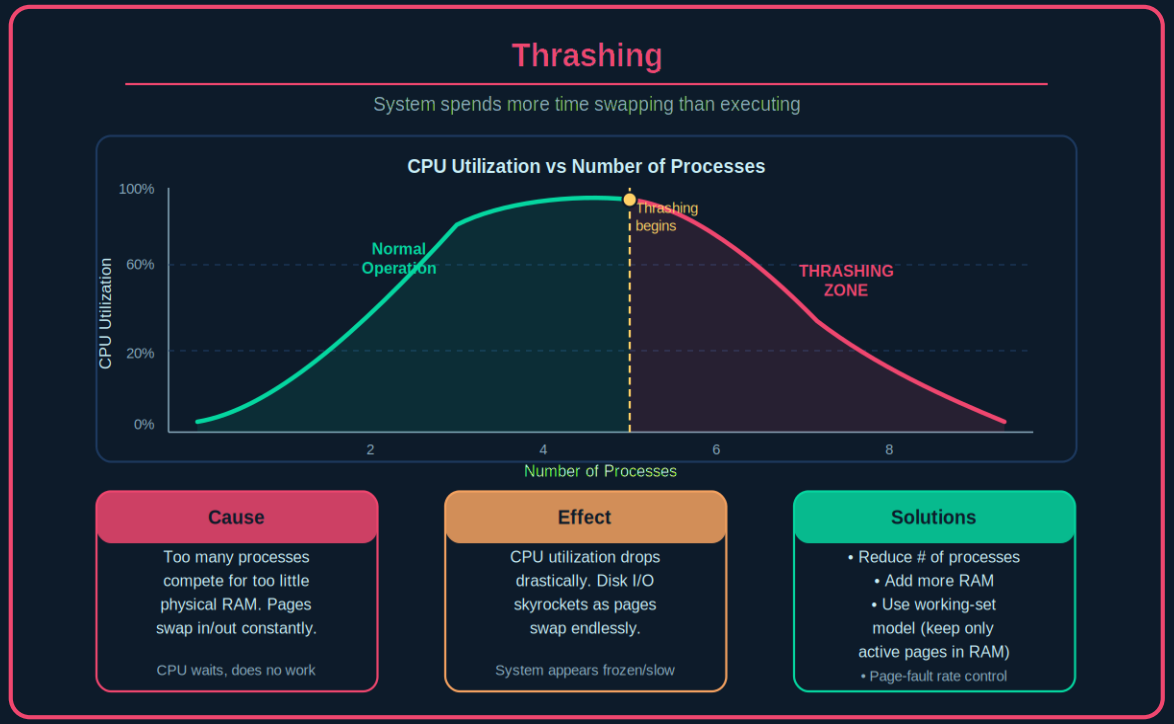
Swap Space

A dedicated disk partition or file reserved as overflow storage for RAM.

Performance

Necessary for multitasking but slow — disk I/O is far slower than RAM access.

13 | Thrashing



What is Thrashing?

System spends more time swapping pages in/out than actually executing processes.

Cause

Too many processes compete for too little RAM. Every process needs pages the other holds.

Effect

CPU utilization crashes. Disk I/O skyrockets. System feels frozen or extremely slow.

Solutions

Reduce processes, add more RAM, or apply working-set model — keep only active pages in RAM.

Thank You!

Topics: Memory • Memory Management • Types • Virtual Memory • Paging • Segmentation • Demand Paging • TLB • Swapping • Thrashing